

Data Processing Assignment

Pandas Data Cleaning & Delta Lake Incremental ETL Pipeline (SCD1 / SCD2)

Author: Vabhrevi
Pandey

Dataset: Superstore Dataset Final
(Kaggle)

Tech Stack: Python, Pandas, Delta Lake (delta-
rs)

Part 1: Data Exploration, Scrubbing & Cleansing

The primary dataset utilized for this phase was the **Superstore Dataset Final** sourced from Kaggle (by user *vivek468*). The initial raw input contained **10,800 rows across 21 columns**. Detailed structural analysis revealed that the final 806 rows were non-genuine order entries—leftover rows from auxiliary sheets (such as *People* and *Returns* tabs) that had been mistakenly concatenated into the main Orders CSV file. These corrupted rows were identified through non-numeric Row ID values and systematically pruned.

10,800 RAW ROWS	806 CORRUPTED DROPPED	11 POSTAL CODES FIXED	0 EXACT DUPLICATES	9,994 CLEAN FINAL ROWS
---------------------------	------------------------------------	------------------------------------	---------------------------------	-------------------------------------

Following row-level sanitation, missing value imputation was performed on 11 records lacking a **Postal Code** by deriving the statistical mode (most frequent postal code) within their respective cities. Standard data type coercions were applied (converting text strings to standard datetime objects). Additionally, feature engineering was carried out to establish two key derived columns: `unit_price` and `total_amount`, which were verified for integrity against original sales figures.

Data Pipeline Metric	Raw State	Cleaned State	Status / Transformation
Total Row Count	10,800	9,994	-806 invalid entries
Column Count	21	23	+2 derived columns
Missing Postal Codes	11 missing	0 missing	Imputed via City Mode
Exact Duplicate Rows	0	0	Verified Clean

Part 2: Incremental Load Strategies via Delta Lake

To evaluate real-world Data Warehousing update patterns, a synthetic customer dataset was designed to simulate production incremental ETL. The initial master dataset contained **25 customers** (deliberately seeded with 2 duplicate entries and 3 missing phone numbers). An incoming incremental batch of **10 records** was introduced, consisting of **5 updates** to existing customer profiles and **5 brand-new** customer records.

The pipeline leveraged **Delta Lake** via the lightweight `deltalake (delta-rs)` engine, providing native ACID transactions, transaction logging, and Parquet storage without requiring PySpark or JVM dependencies. Equivalent PySpark SQL expressions were fully documented alongside every operation.

Architectural Comparison: SCD Type 1 vs. SCD Type 2

- **Slowly Changing Dimension (SCD) Type 1 — Overwrite Strategy:** Uses a standard single-pass `MERGE INTO` clause (`WHEN MATCHED UPDATE / WHEN NOT MATCHED INSERT`). It overwrites historical data to maintain strictly one current row per customer (30 total rows after processing).
- **Slowly Changing Dimension (SCD) Type 2 — Historical Audit Trail:** Implements a robust two-stage merge pattern. First, it expires modified active records by setting `is_current = false`. Next, it inserts fresh active rows for both modified and brand-new customers. This produces 35 total rows (30 active + 5 historical versions), maintaining an immutable, fully queryable audit history.

Execution Metric	SCD Type 1 (Overwrite)	SCD Type 2 (Versioned)	Delta Lake Log Note
Total Table Rows After Merge	30	35	Includes 5 historical record versions in SCD2
Active / Current Rows	30	30	Identical active state across both methods
Historical Audit Rows	0	5	Preserved prior versions (<code>is_current = false</code>)
Duplicate Customer IDs	0	N/A (by design)	SCD2 retains multiple versions per ID

Verification, Transaction Log & Deliverables

All row count assertions, primary key constraints, and duplicate validations passed programmatically within the automated notebook suite. Furthermore, inspection of the Delta Lake transaction log for the SCD Type 2 table confirmed proper ACID commit sequencing:

```
Delta Transaction Log Commit Sequence:
└─ v0: WRITE           # Initial table initialization with base master data
└─ v1: MERGE           # Expiration pass for modified customer profiles (is_current = false)
└─ v2: MERGE           # Insertion pass for new active records and incremental updates
```

Project Repository Artifacts & Files:

```
└─ notebooks/
  │ └─ 01_pandas_data_cleaning.ipynb    (Part 1: Superstore ETL)
  │ └─ 02_delta_lake_incremental.ipynb  (Part 2: SCD1 & SCD2 Delta MERGE)
└─ data/                                (Raw & cleaned datasets for both tasks)
└─ screenshots/                         (Stage-by-stage execution evidence)
└─ README.md                           (Project setup, dependencies & documentation)
```